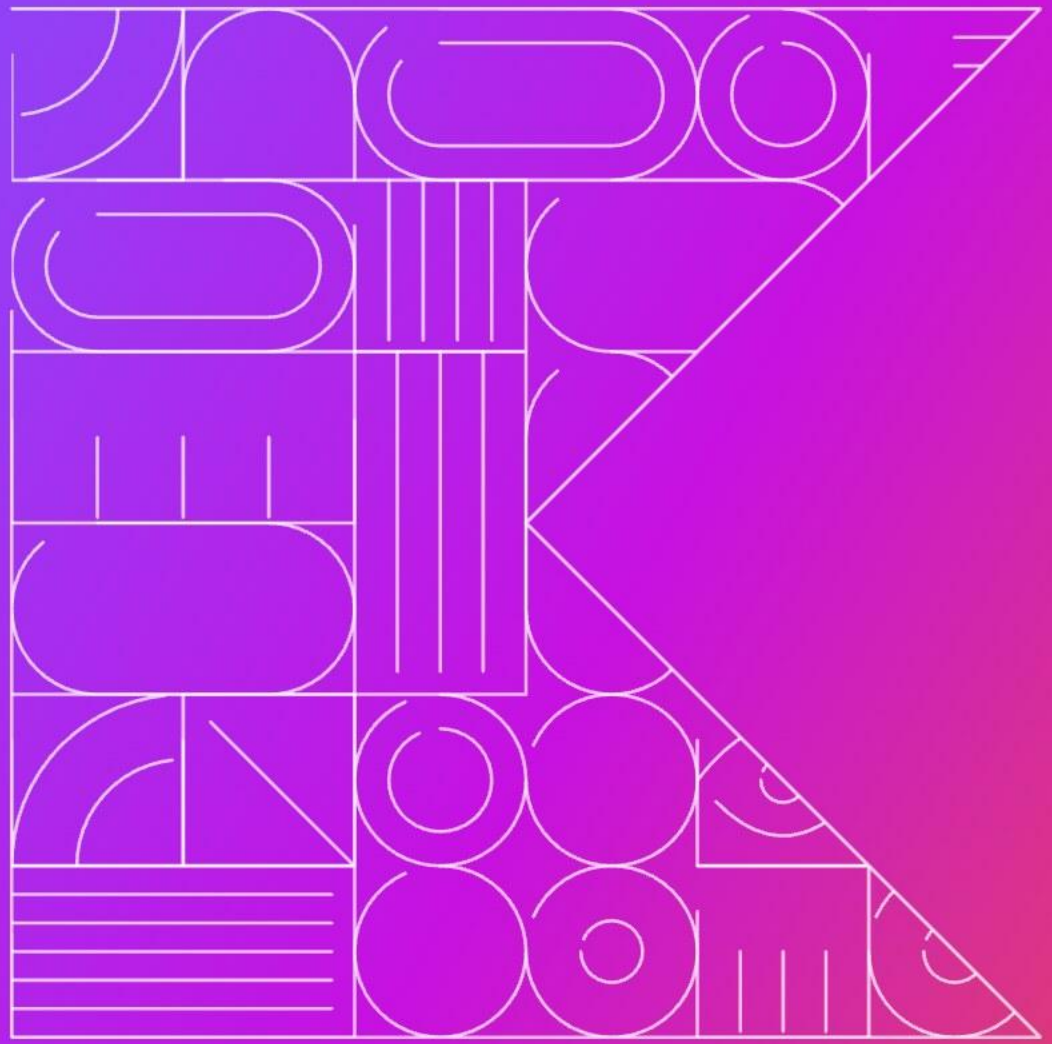
DevFest Chennai 2026

Kotlin Over the Air

Ship a Compose screen to users in minutes. No store release.



Aditya Shinde
Founder – Ketoy.dev

Follow along

These slides, on your phone.

`slides.ketoy.dev/devfest-chennai-26`

Scan now. Everything on stage is in your pocket.



Your fix is ready today.
Your users get it next week.



Fix merged
Day 0

Build + QA
Day 1–2

Store review
hours to 2 days

Staged rollout
days

Users update
1–2 weeks · some never

Server-driven UI in production	**Airbnb · Lyft · Netflix · Spotify**
Code over the air in production	**React Native (Expo Updates) · Flutter (Shorebird)**

Kotlin and Compose?

October 3, 2026

a week in review



Peter Steinberger @steipete · Oct 3

Do I know anyone at Google who could help? We're now over a week in review limbo for OpenClaw's Android app.

217 227 8.8K 1.7M



Sundar Pichai
@sundarpichai

Follow

Ack, will follow up

12:08 AM · Oct 4, 2026 · **6.3M** Views

Do the math

3 updates a month × **2** days of waiting each = **6** days lost a month

72 days a year.



a fix that's ready, waiting for your users · about 1 day in 5

That's what your team is losing.

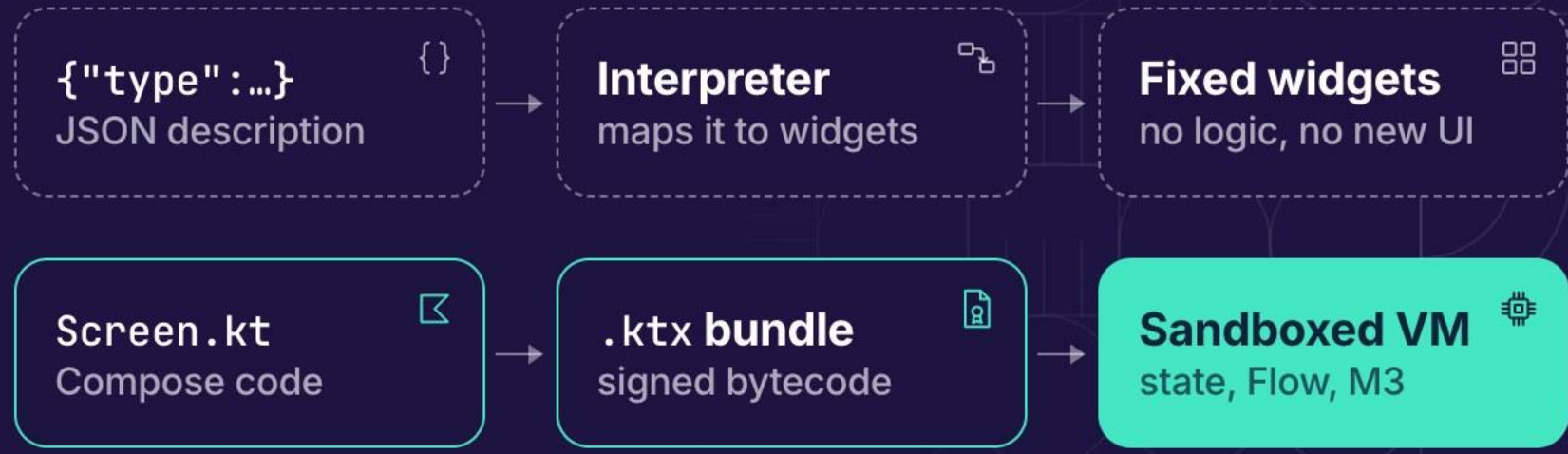
Your Diwali banner has to be live on Diwali morning.

Not when the last user finally updates.

Describing a screen vs shipping a program

Server-driven UI
sends a description

Kotlin over the air
sends a program



SDUI changes what a screen looks like.
OTA changes what it does.

Both run only what the app chose to expose. We call those **capabilities**.

Server-driven UI, end to end

The server sends JSON. The app ships the interpreter.

`promo.json` sent by the server

```
{
  "type": "column",
  "padding": 16,
  "children": [
    { "type": "text", "text": "Diwali Sale" },
    { "type": "text", "text": "Flat 40% off" },
    { "type": "button", "text": "Shop now",
      "action": "navigate:/offers" }
  ]
}
```

`Render.kt` compiled into the app

```
@Composable
fun Render(n: Node) {
  when (n["type"]) {
    "column" -> Column(
      Modifier.padding(n.dp("padding"))
    ) { n.children.forEach { Render(it) } }
    "text"   -> Text(n["text"])
    "button" -> Button(onClick = {
      perform(n["action"])
    }) { Text(n["text"]) }
    else -> {} // unknown type: nothing shows
  }
}
```

Every `"type"` the app doesn't know renders nothing. A new component, or any logic, means a new app release.

Kotlin over the air, end to end

The same card, as plain Kotlin.

`DiwaliPromo.kt` what you write

```
@KetoyComposable
fun DiwaliPromo() {
  Column(Modifier.padding(16.dp)) {
    Text("Diwali Sale")
    Text("Flat 40% off")
    Button(onClick = { navPush("/offers") }) {
      Text("Shop now")
    }
  }
}
```

`KBC` what the compiler emits

```
fn0  DiwaliPromo
  COMPOSABLE_CALL Column modifier=#0 content=fn1
  RETURN
fn1  Column { … }
  COMPOSABLE_CALL Text text="Diwali Sale"
  COMPOSABLE_CALL Text text="Flat 40% off"
  COMPOSABLE_CALL Button onClick=fn2 content=fn3
  RETURN
fn2  onClick
  LOAD_STRING r0 "/offers"
  INVOKE_CAPABILITY r1 NAV_PUSH [r0]
  RETURN
fn3  Button { … }
  COMPOSABLE_CALL Text text="Shop now"
  RETURN
```

Kotlin over the air, end to end

…compiled into one signed file.

KBC what the compiler emits

```
fn0  DiwaliPromo
  COMPOSABLE_CALL Column modifier=#0 content=fn1
  RETURN
fn1  Column { … }
  COMPOSABLE_CALL Text text="Diwali Sale"
  COMPOSABLE_CALL Text text="Flat 40% off"
  COMPOSABLE_CALL Button onClick=fn2 content=fn3
  RETURN
fn2  onClick
  LOAD_STRING r0 "/offers"
  INVOKE_CAPABILITY r1 NAV_PUSH [r0]
  RETURN
fn3  Button { … }
  COMPOSABLE_CALL Text text="Shop now"
  RETURN
```

main.ktx what you ship

HEADER	`4B 54 4F 59 …` "KTOY" · version · flags
STRINGS 0x01	`00 0B 44 69 77 61 6C 69 20 53 61 6C 65` "Diwali Sale" · u16 length + UTF-8
CODE 0x07	`B0 00 01 01 00 05 00 06` COMPOSABLE_CALL Text "Shop now"
	`08 00 00 04 50 01 07 00 01 00` LOAD_STRING "/offers" · NAV_PUSH
	`78 9C …` zlib-compressed on disk
SIGNATURE	`64 bytes` Ed25519, checked before anything is read

The app is the operating system.
The server ships programs.



From `.kt` to a phone in five steps



01
Source
Kotlin + Compose

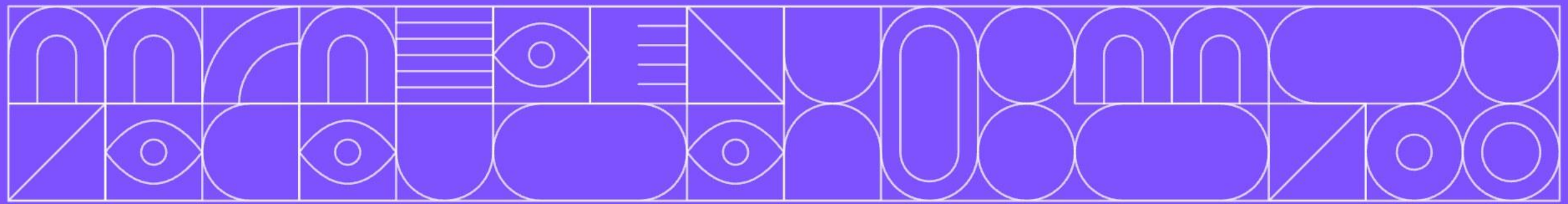
02
Compiler
checks the sandbox

03
.ktx
a signed file

04
VM
inside your app

05
Compose
native UI

KBC = Ketoy bytecode, our own small instruction set.



The compiler

Plain Kotlin in. Bytecode out.

```
@KetoyComposable
fun TodoListScreen() {
  val todos by observeTodos().collectAsState(emptyList())
  LazyColumn(Modifier.padding(16.dp)) {
    items(todos) { todo ->
      Text(todo.title, fontWeight = FontWeight.Bold)
    }
  }
}
```

One annotation.

Everything this screen reaches compiles to KBC. Everything else stays normal Android code.

The compiler

Plain Kotlin in. Bytecode out.

```
@KetoyComposable
fun TodoListScreen() {
  val todos by observeTodos().collectAsState(emptyList())
  LazyColumn(Modifier.padding(16.dp)) {
    items(todos) { todo ->
      Text(todo.title, fontWeight = FontWeight.Bold)
    }
  }
}
```

Real state and Flow.

`collectAsState`, `remember` and `ViewModel` run in the VM, with real recomposition.

The compiler

Plain Kotlin in. Bytecode out.

```
@KetoyComposable
fun TodoListScreen() {
  val todos by observeTodos().collectAsState(emptyList())
  LazyColumn(Modifier.padding(16.dp)) {
    items(todos) { todo ->
      Text(todo.title, fontWeight = FontWeight.Bold)
    }
  }
}
```

Real Compose calls.

Parameters are matched by name, so a Material3 upgrade doesn't break old bundles.

The compiler

Plain Kotlin in. Bytecode out.

```
@KetoyComposable
fun TodoListScreen() {
  val todos by observeTodos().collectAsState(emptyList())
  Toast.makeText(context, "hi", Toast.LENGTH_SHORT).show()
}
```

This doesn't compile.

The build fails, and the message names the fix.

```
e: KetoyBC: Direct access to android.widget.Toast. Expose a capability instead.
```

The compiler

Your app moved to AGP 9. The bundle didn't notice.



The KBC compiler ships its own Kotlin (2.4.10). Your app's Kotlin and AGP versions don't matter.

What it costs you

Two files. One annotation.



app/build.gradle.kts

```
plugins {
  alias(ketoy.plugins.compiler)
}
dependencies {
  implementation(ketoy.bundles.core)
  implementation(ketoy.annotations)
}
ketoy {
  exportFromAppModule.set(true)
}
```

AboutScreen.kt

```
@KetoyComposable
fun AboutScreen() {
  Text("Vanakkam!")
}

// in your NavHost
KetoyScreen(
  entryPoint = "AboutScreen",
  nativeFallback = { AboutNative() },
)
```

```
./gradlew :app:ketoyBundle → assets/ketoy/main.ktx
```

LIVE

One command, a few minutes, and it's live.

```
$ ketoy push ktx main.ktx --version 12
```

Messed up? No worries. Roll back instantly.

```
$ ketoy ktx rollback
```



The .ktx file

A program you can ship



Checked first, against a key inside your APK

1.6 KB
a real screen on Google Play

≈0.7 MB
the whole SDK in your APK

0
bytes parsed before the signature verifies

On the phone

Verified before it's even read

01
Verify signature
key in the APK

02
Read sections
version check

03
Check needs
every capability exists, by name

04
Activate
or fall back to the native screen

One byte changed? It stops at step 1 and shows your native screen.



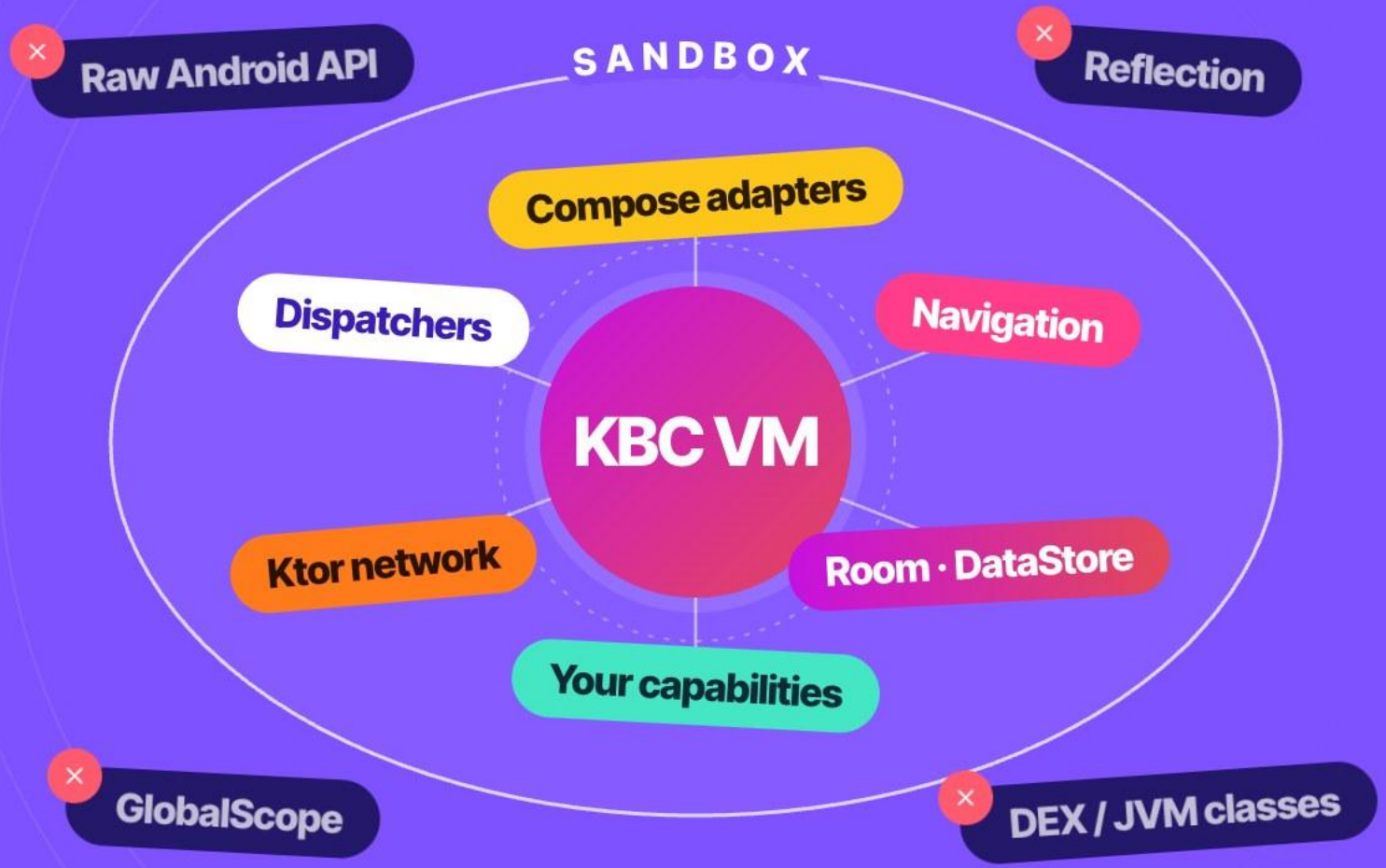
Something we got wrong

Every bundle we shipped was Brotli.

Brotli decoding needed a JVM library, so our "portable" format was Android-only. Now it's DEFLATE: zlib exists everywhere.



A VM that can only reach what you hand it



Real coroutines
suspend, Flow, cancellation

One VM per process
state survives rotation

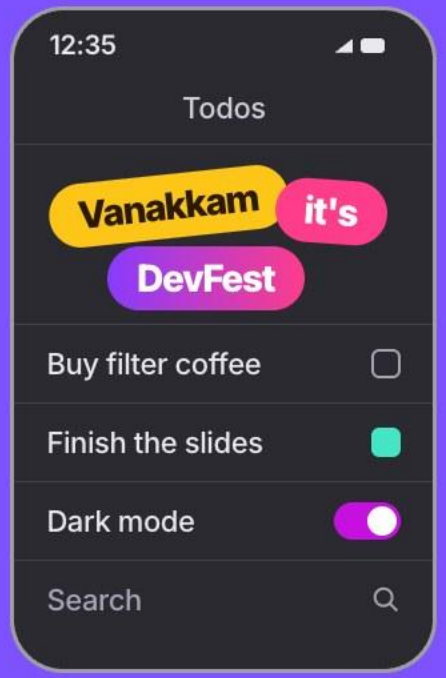
Nothing executable
no reflection, no JIT on iOS

Host registers `observeTodos`. Bundle declares it needs `observeTodos`. Missing? **Rejected at load, by name.**

One signed bundle. Three renderers.

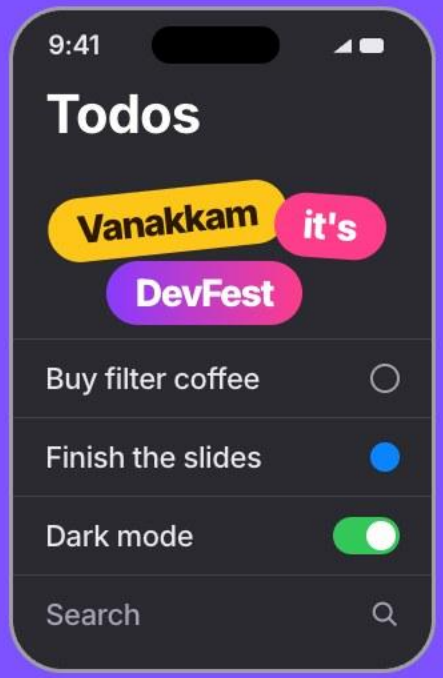
`main.ktx · 7.6 KB · one signing key`

Android Compose



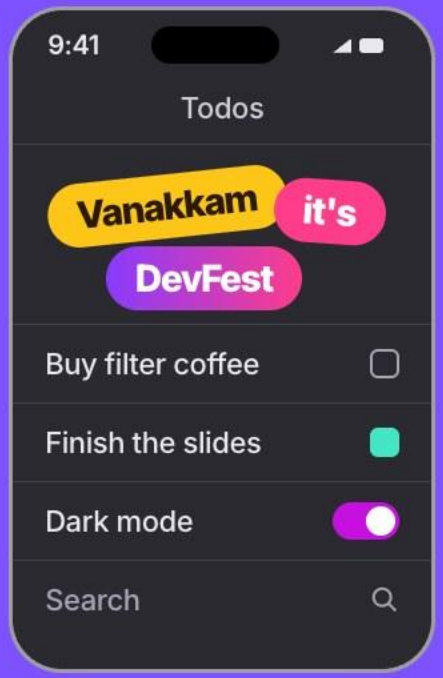
Live on Google Play

iOS SwiftUI



Real VStack and Text

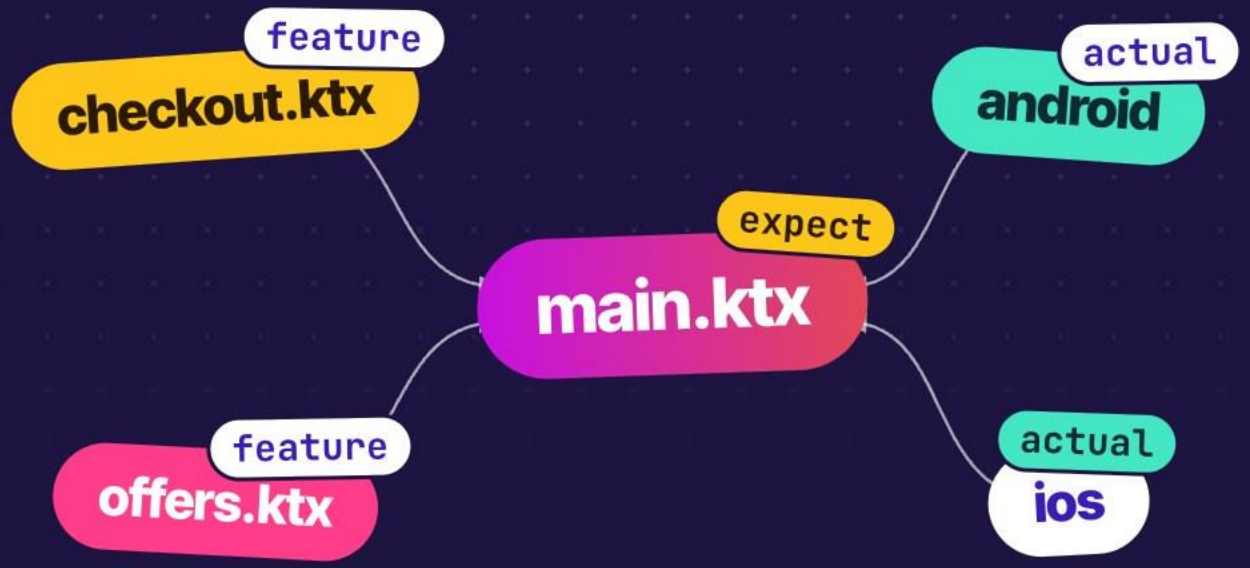
iOS Compose MP



Material3 on Skia, 30 of 30

At scale

Many bundles. One signed graph.



Canary
1% → 25% → 100%, per device

Kill switch
pin a bundle, revoke a key

Automatic rollback
a crash returns to the last good one

Native fallback, always
bad or missing: your native screen

`expect` / `actual`, at the bundle boundary. Signed and switched on as one.

No control plane. Your CDN, your remote config, your rules.

Honestly

When not to use this

Only need copy and layout?	Use SDUI or remote config. It's simpler.
Need a platform API the app doesn't expose?	That's an app release. By design.
`Canvas` **on the SwiftUI backend?**	Not yet. It fails by name, not silently.



If you remember three things

1 The app is the OS. The server ships programs.

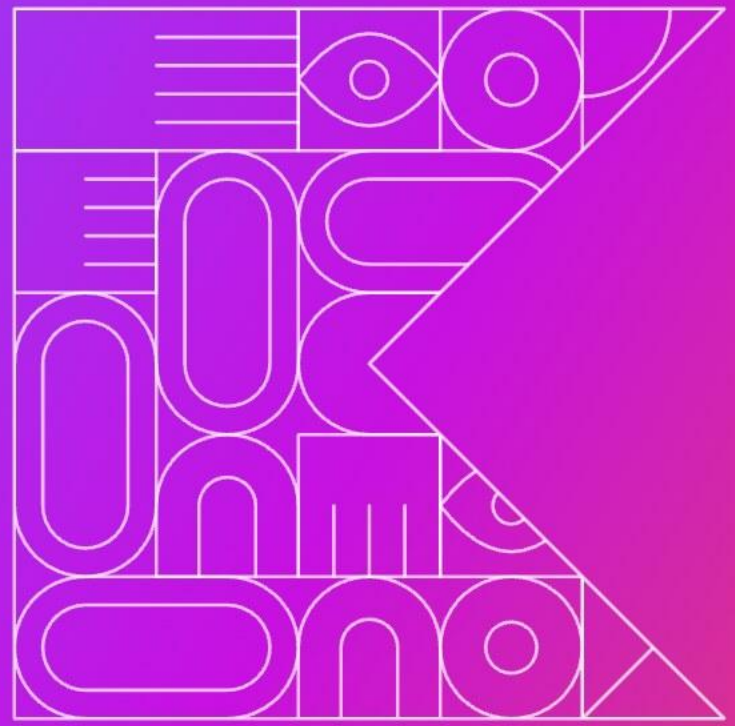
2 One annotation. The compiler is the sandbox.

3 Signed before parsed. Native fallback, always.

Your fix is ready today. Your users get it today.



Thank you



Let's connect

linkedin.com/in/adityashinde1095

ketoy.dev

Aditya Shinde
Founder – Ketoy.dev

#DevFestChennai